Практика 1

Создание и запуск модульных тестов для управляемого кода

Гонцов Арсений

ИПс-21

Оглавление

Создание основного проекта 3

Создание проекта модульного теста 4

Создание тестового класса 5

Создание первого тестового метода 6

Сборка и запуск теста 7

Исправление кода и повторный запуск 8

тестов 8

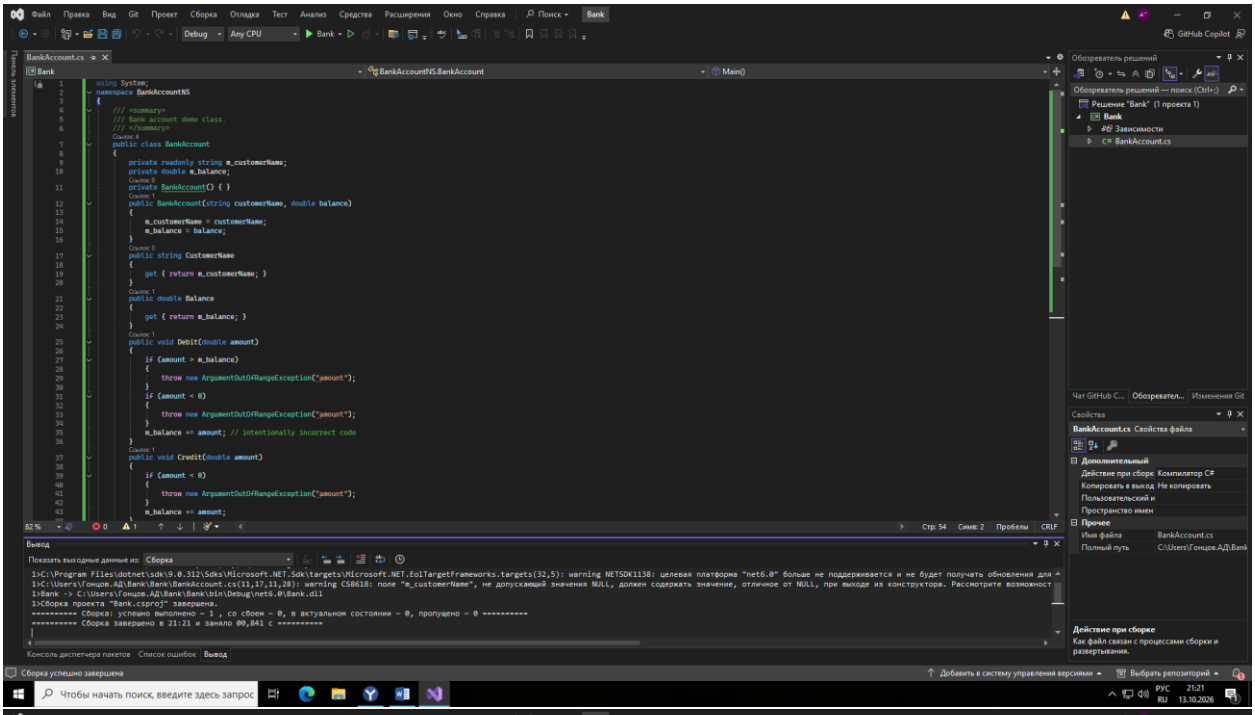
Повторный запуск теста 9

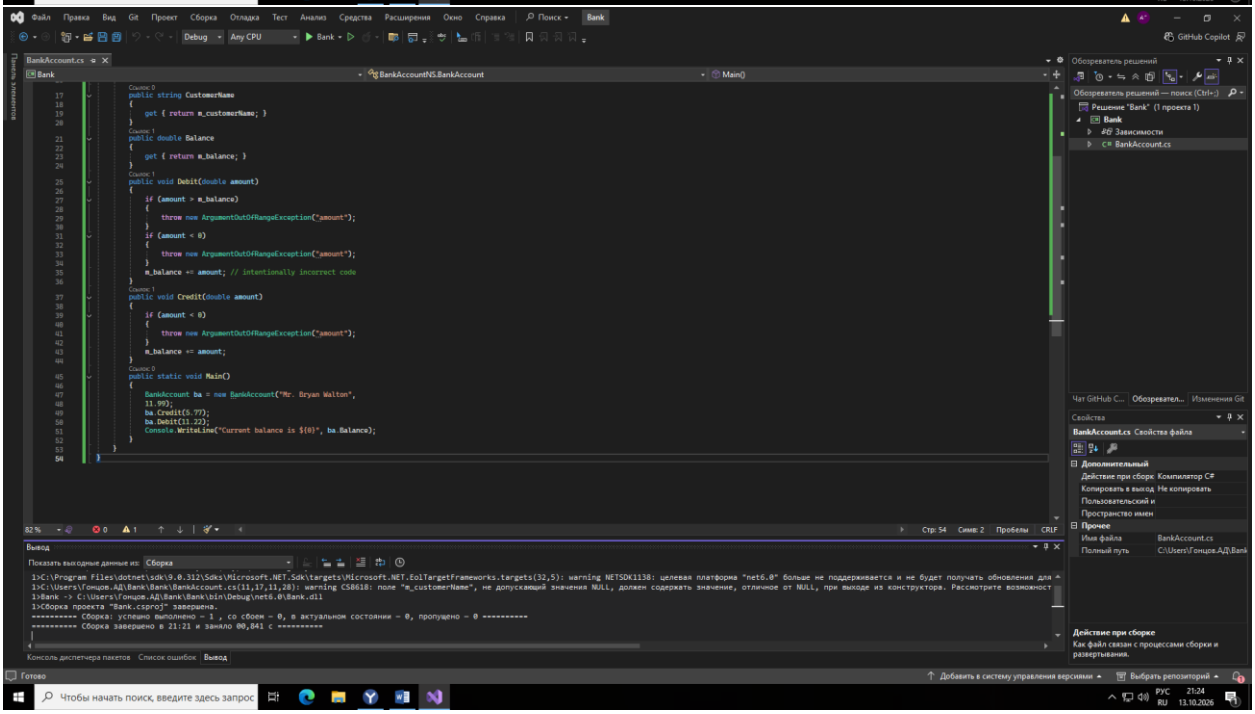
Создание и запуск новых методов теста 10

Рефакторинг тестируемого кода 11

Рефакторинг тестовых методов 12

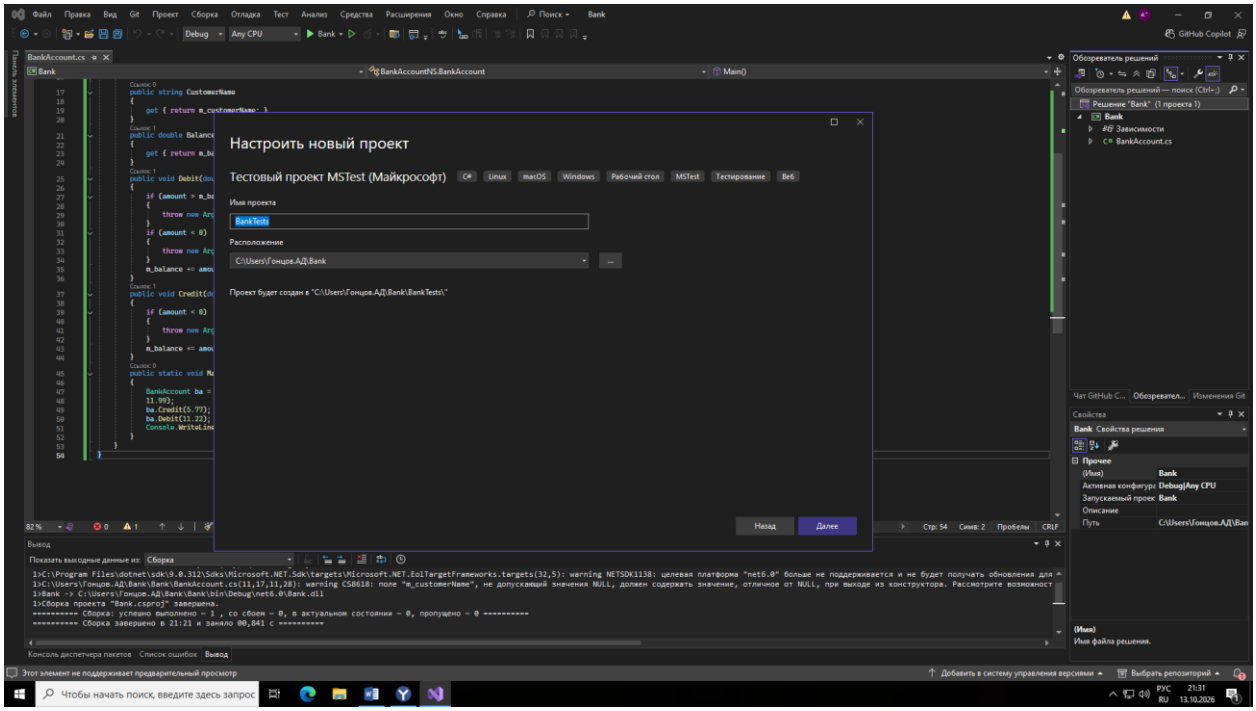
Создание основного проекта



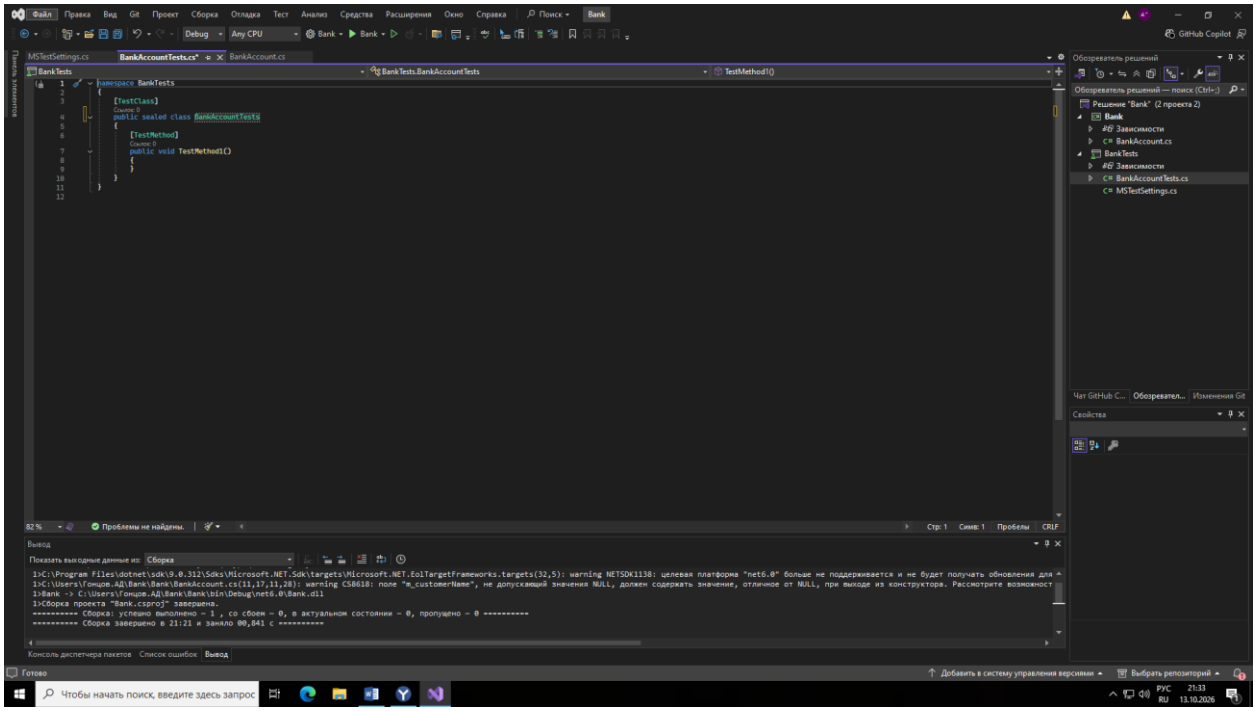


Был создан проект, проекту было присвоено имя Bank, в проекте создан основной класс банковского счёта BankAccountNS.cs и вписан данный мне код.

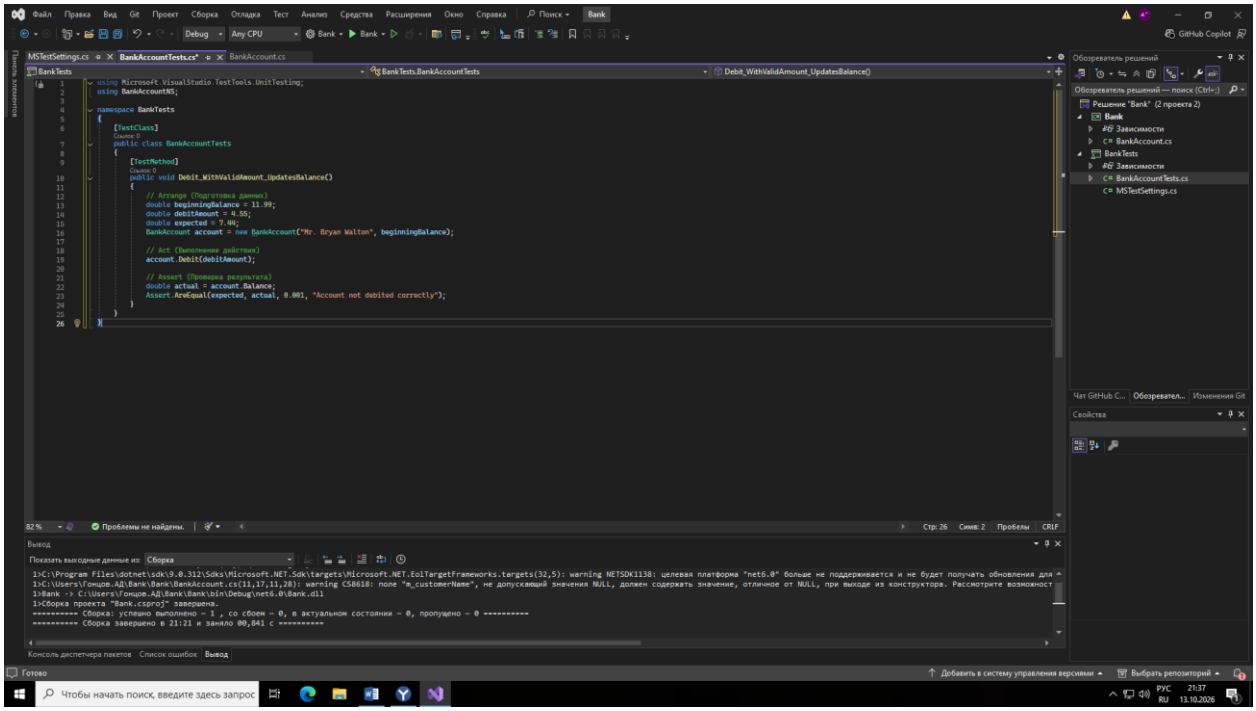
Создание проекта модульного теста



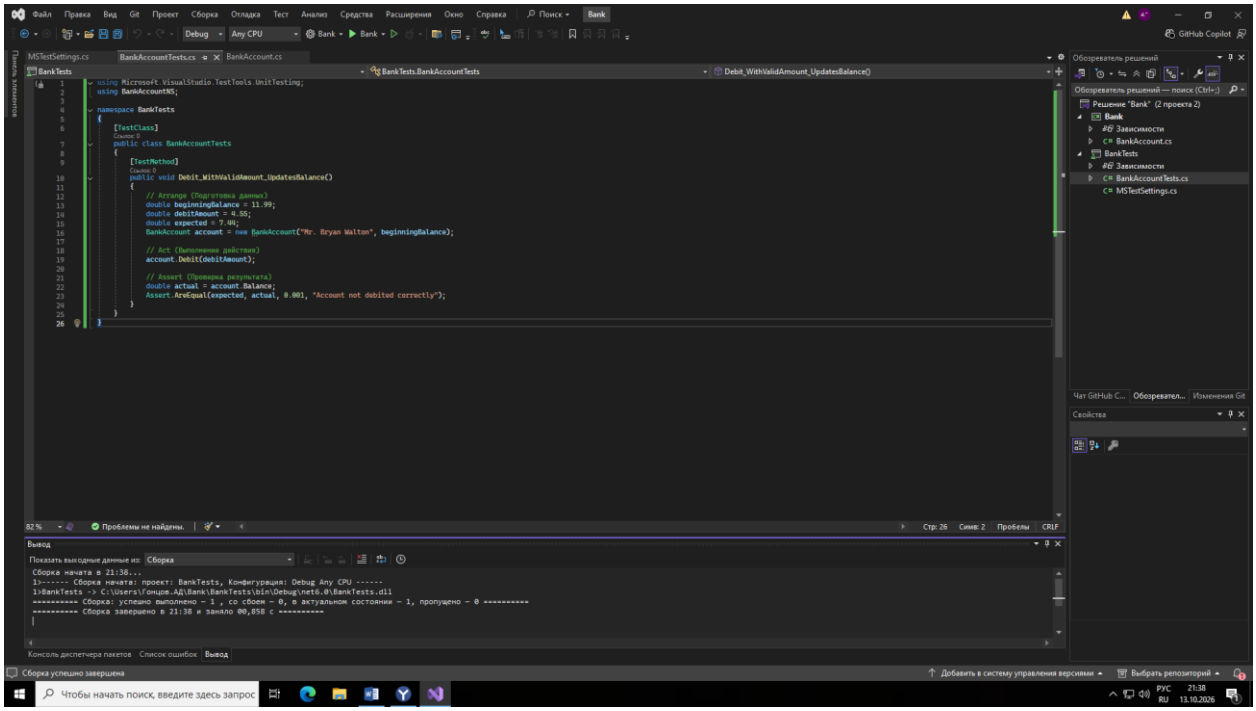
Создание тестового класса



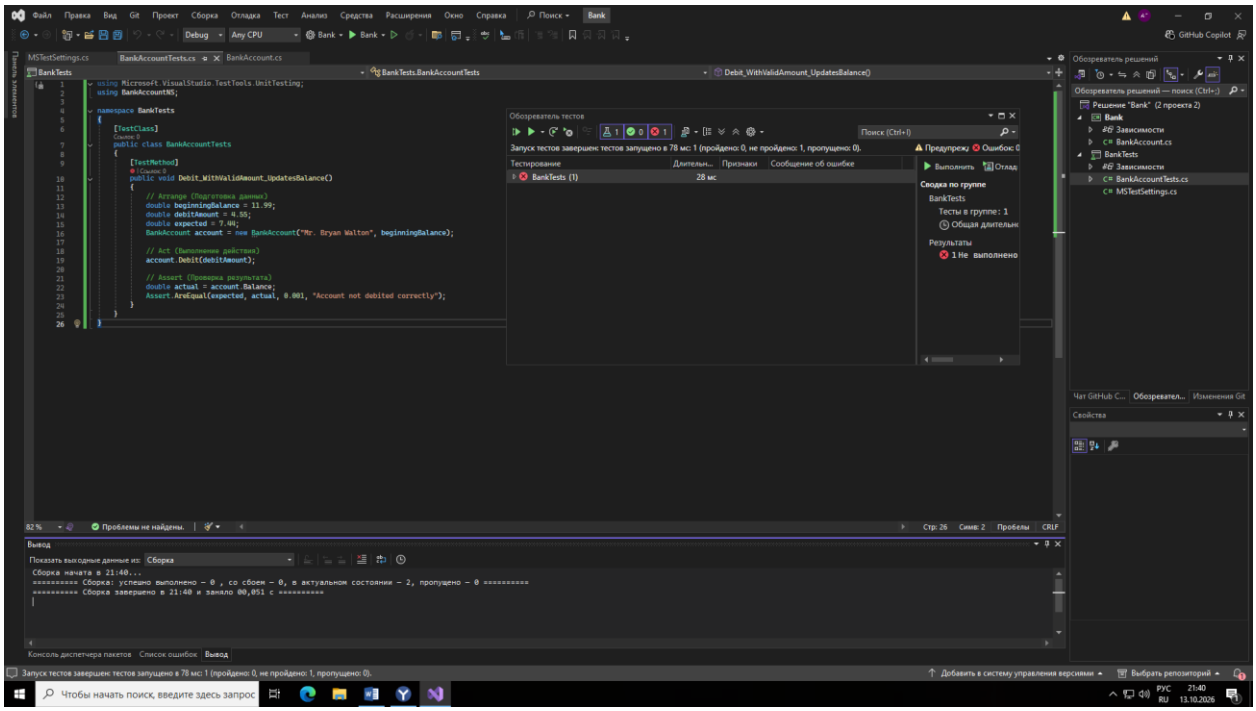
Создание первого тестового метода



Сборка и запуск теста

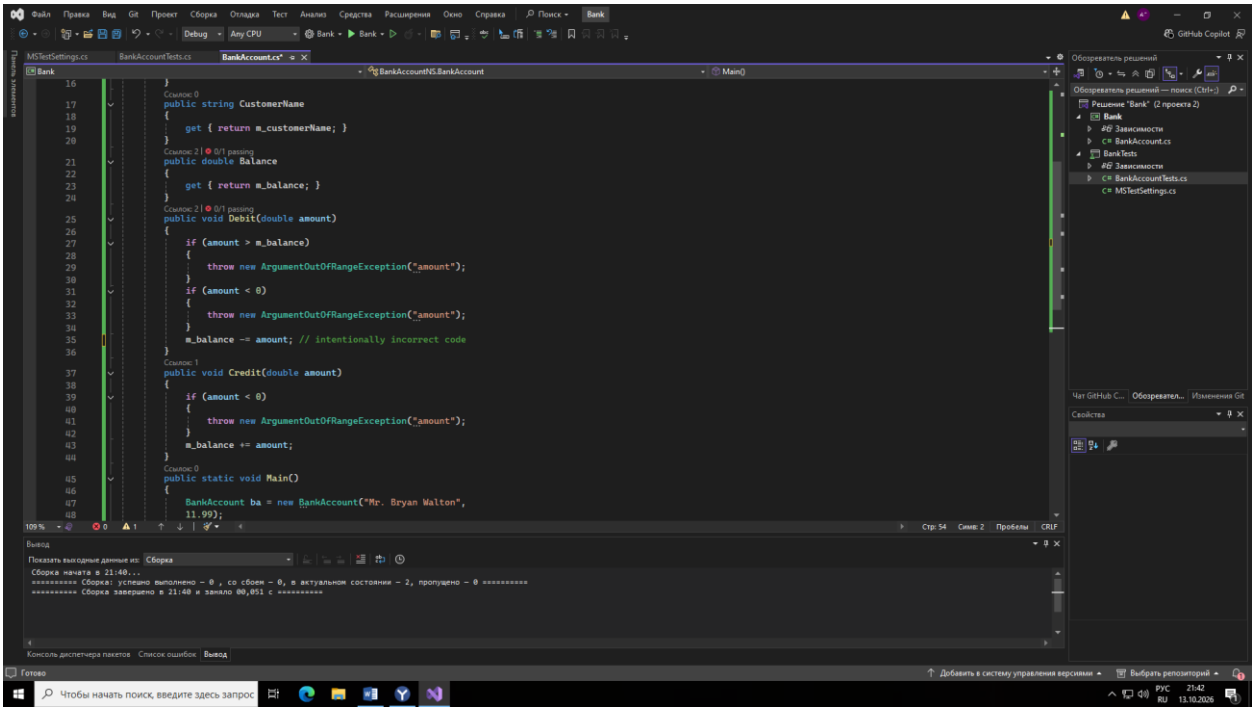


Сборка прошла успешно



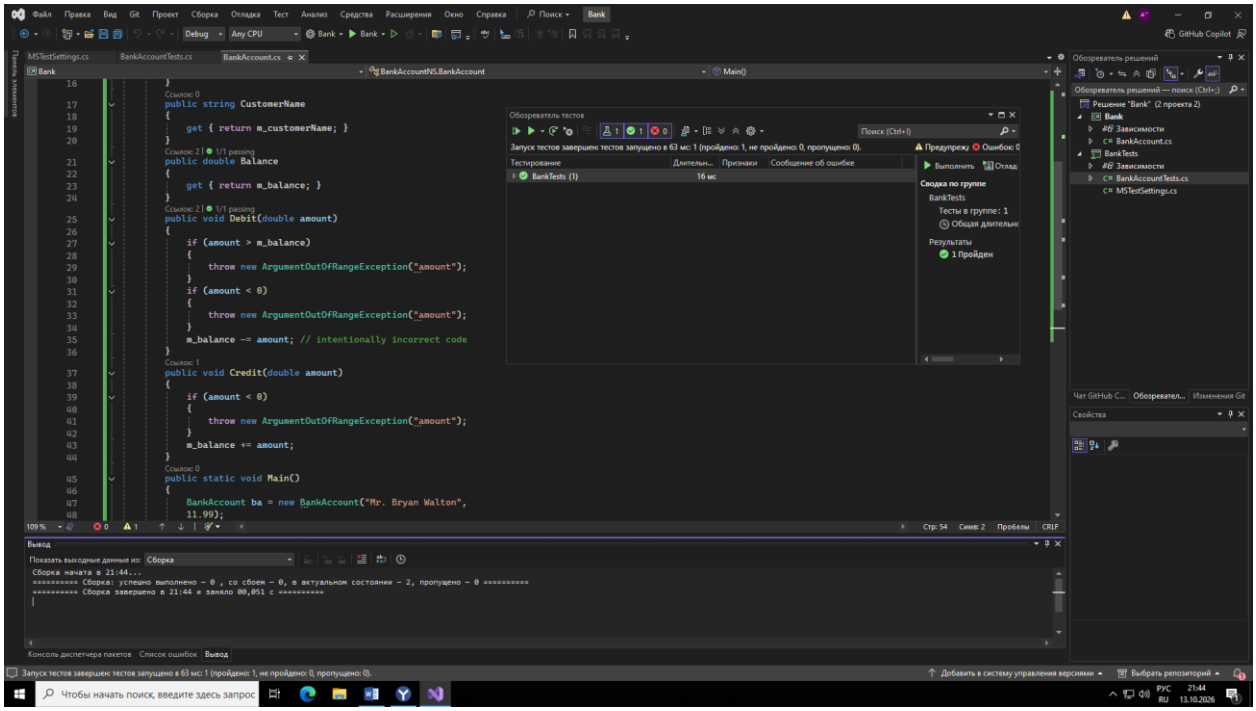
Тест не пройден

Исправление кода и повторный запуск тестов



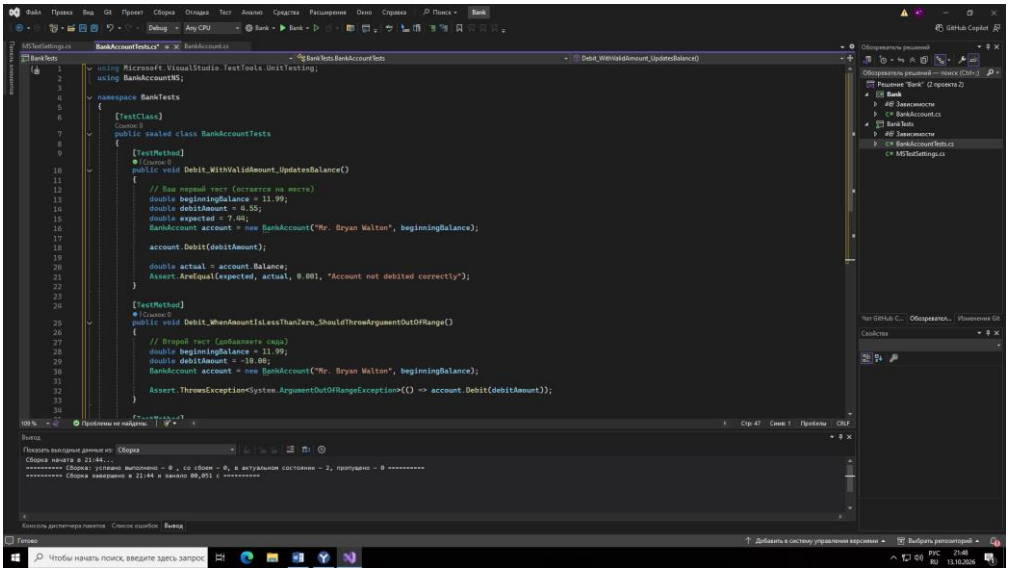
Исправлен код

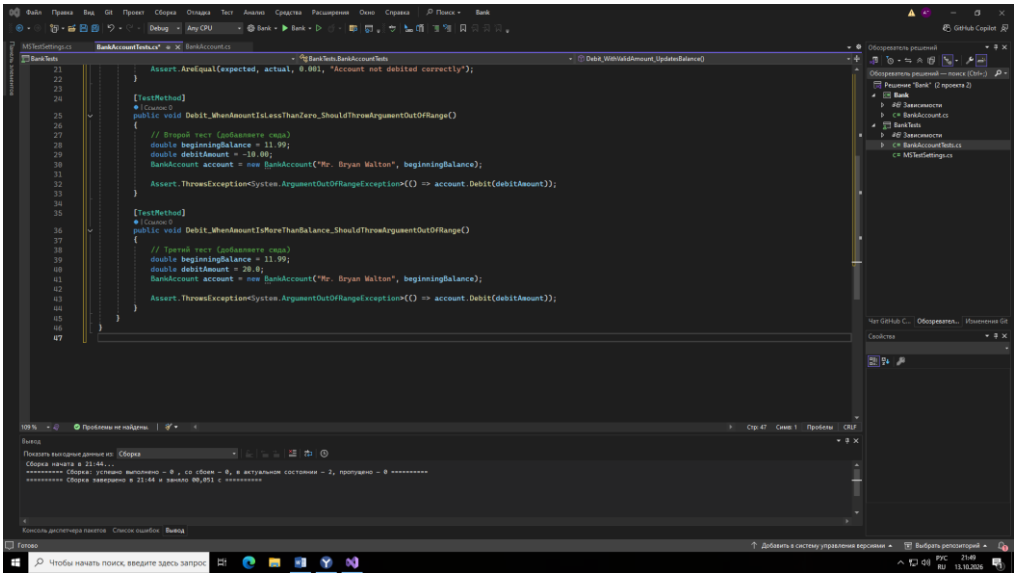
Повторный запуск теста

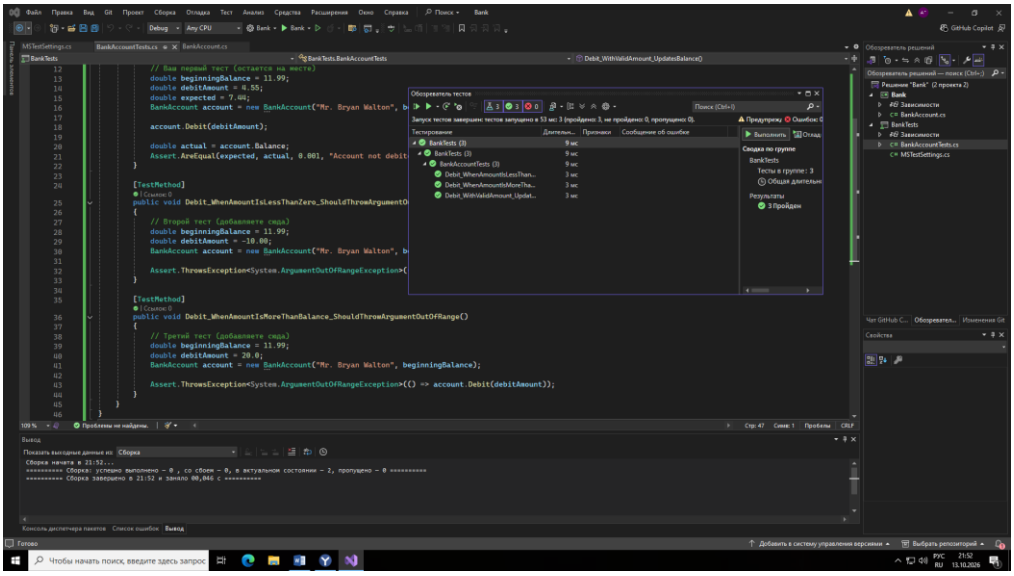


Тест пройден

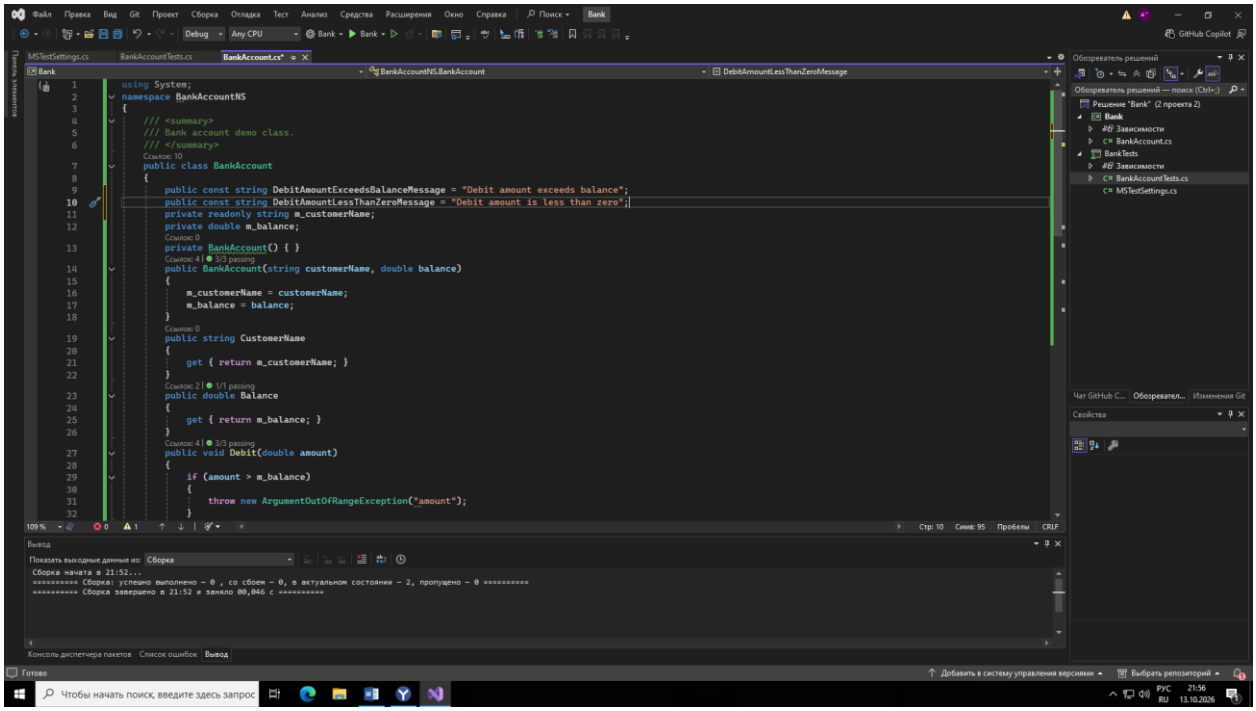
Создание и запуск новых методов теста



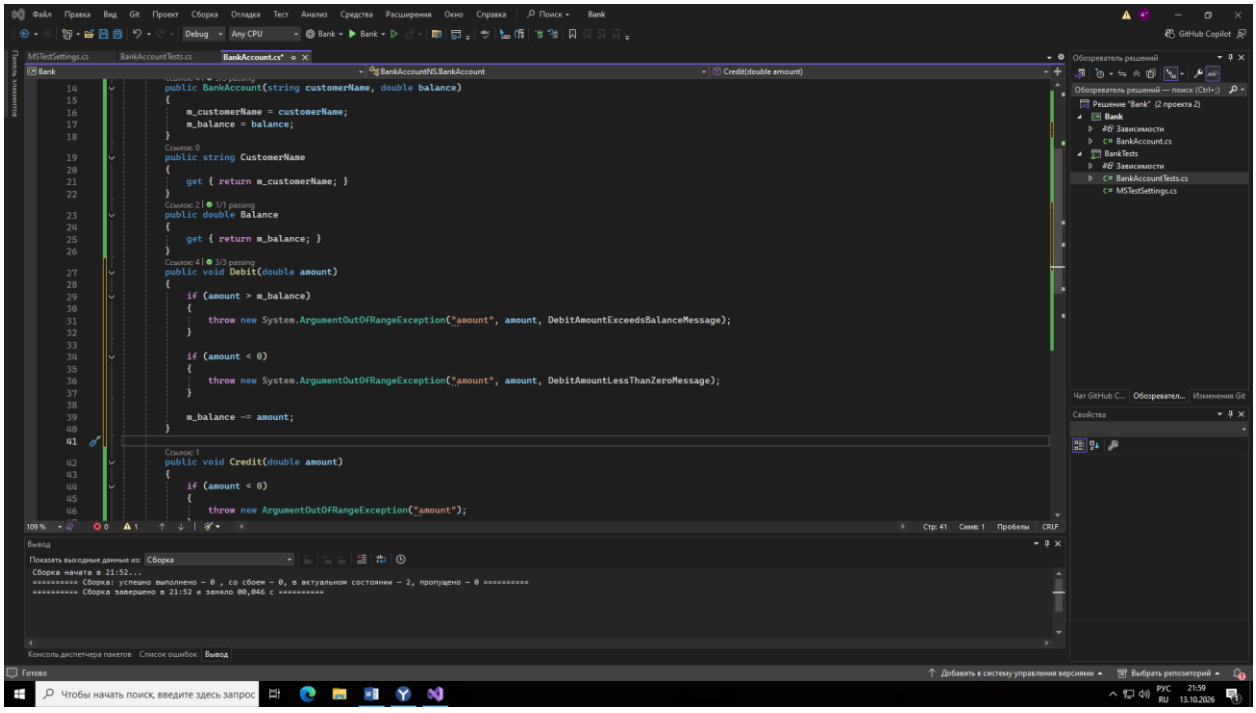




Рефакторинг тестируемого кода

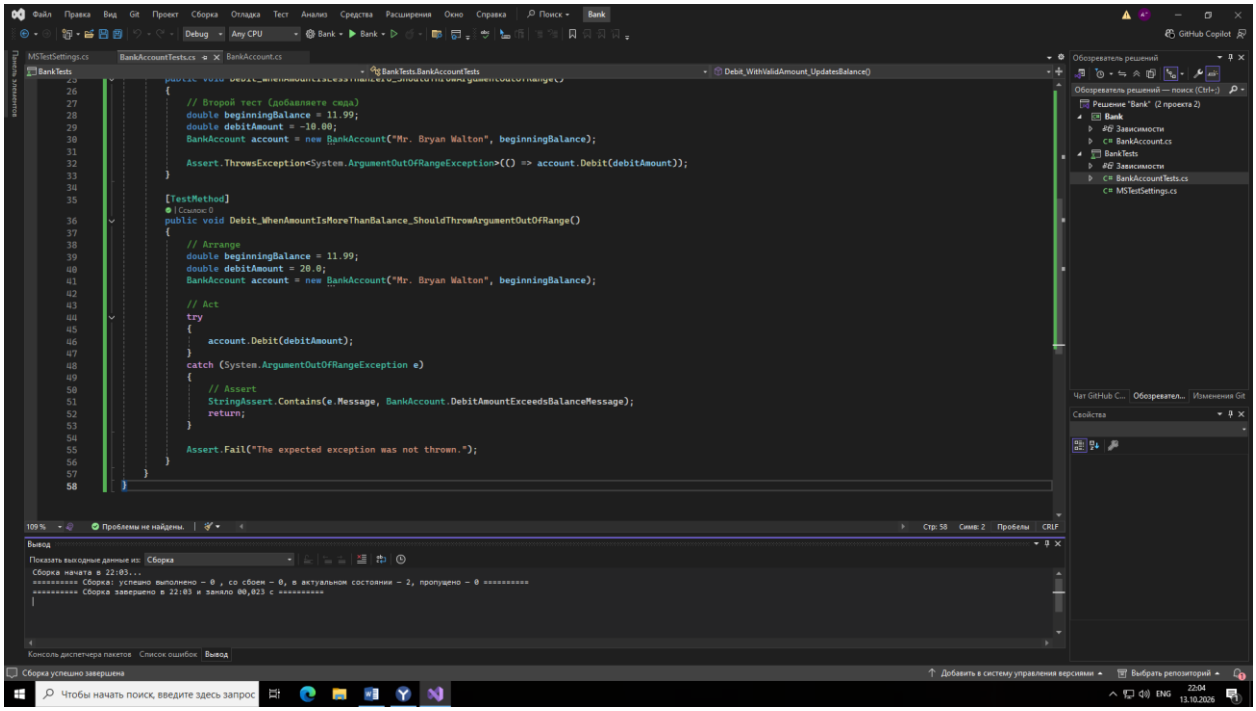


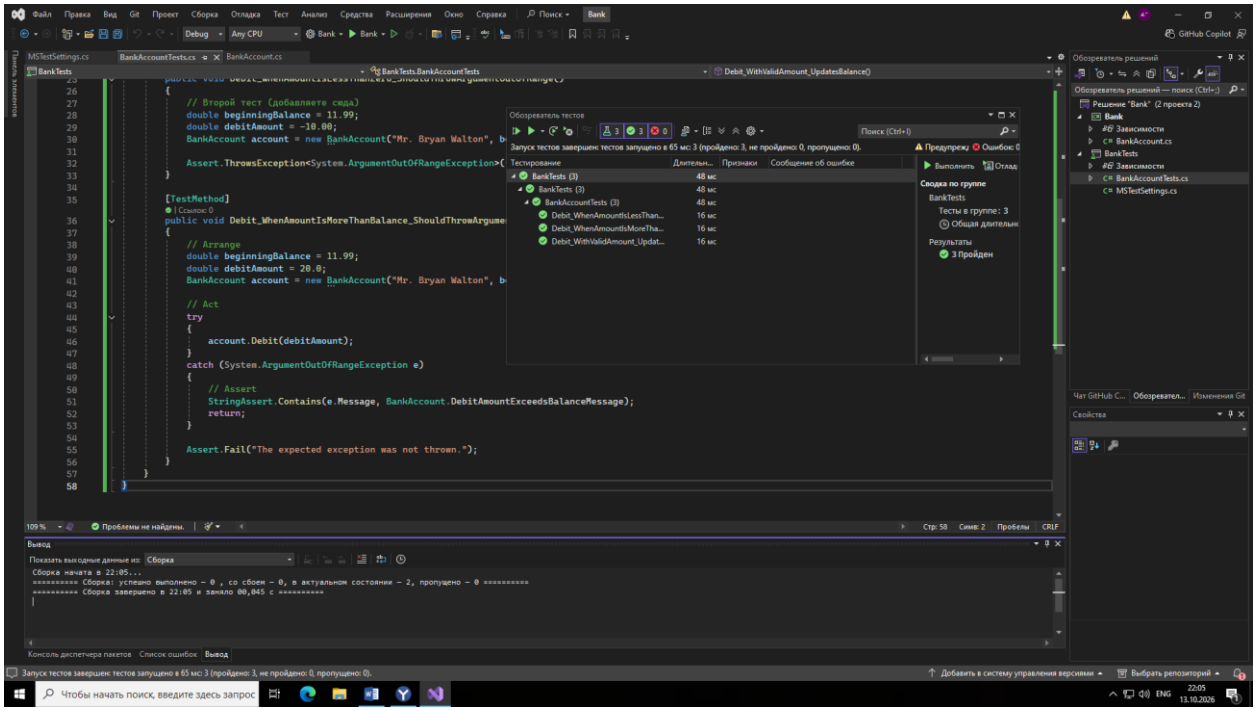
Добавил две константы для сообщений об ошибках в области видимости класса.



изменил два условных оператора в методе Debit

Рефакторинг тестовых методов





В ходе выполнения практической работы были успешно освоены базовые механизмы модульного тестирования в среде Visual Studio с использованием фреймворка MSTest. В результате работы был выявлен дефект в логике программы, выполнено его исправление, а также проведен рефакторинг исходного и тестового кода для повышения качества и надежности программного продукта. Все 3 написанных модульных теста успешно проходят проверку.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using BankAccountNS;

namespace BankTests
{
    [TestClass]
    public sealed class BankAccountTests
    {
        [TestMethod]
        public void Debit_WithValidAmount_UpdatesBalance()
        {
            // Ваш первый тест (остается на месте)
            double beginningBalance = 11.99;
            double debitAmount = 4.55;
            double expected = 7.44;
            BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);

            account.Debit(debitAmount);

            double actual = account.Balance;
            Assert.AreEqual(expected, actual, 0.001, "Account not debited
correctly");
        }

        [TestMethod]
        public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
        {
            // Второй тест (добавляете сюда)
            double beginningBalance = 11.99;
            double debitAmount = -10.00;
            BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);

            Assert.ThrowsException<System.ArgumentOutOfRangeException>(() =>
account.Debit(debitAmount));
        }

        [TestMethod]
        public void
Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange()
        {
            // Arrange
            double beginningBalance = 11.99;
            double debitAmount = 20.0;
            BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);

            // Act
            try
            {
                account.Debit(debitAmount);
            }
            catch (System.ArgumentOutOfRangeException e)
            {
                // Assert
                StringAssert.Contains(e.Message,
BankAccount.DebitAmountExceedsBalanceMessage);
                return;
            }

            Assert.Fail("The expected exception was not thrown.");
        }
    }
}
```

```
using System;
namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    public class BankAccount
    {
        public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds
balance";
        public const string DebitAmountLessThanZeroMessage = "Debit amount is less
than zero";
        private readonly string m_customerName;
        private double m_balance;
        private BankAccount() { }
        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }
        public string CustomerName
        {
            get { return m_customerName; }
        }
        public double Balance
        {
            get { return m_balance; }
        }
        public void Debit(double amount)
        {
            if (amount > m_balance)
            {
                throw new System.ArgumentOutOfRangeException("amount", amount,
DebitAmountExceedsBalanceMessage);
            }

            if (amount < 0)
            {
                throw new System.ArgumentOutOfRangeException("amount", amount,
DebitAmountLessThanZeroMessage);
            }

            m_balance -= amount;
        }

        public void Credit(double amount)
        {
            if (amount < 0)
            {
                throw new ArgumentOutOfRangeException("amount");
            }
            m_balance += amount;
        }
        public static void Main()
        {
            BankAccount ba = new BankAccount("Mr. Bryan Walton",
            11.99);
            ba.Credit(5.77);
            ba.Debit(11.22);
            Console.WriteLine("Current balance is ${0}", ba.Balance);
        }
    }
}
```